Mathias JANIN
Inès LUNION
Oussama NASRY
Gabriel CLAIRAC-VUITTON

Migration d'une Base de Données Relationnelle vers un Modèle Graphe pour l'Analyse des Crimes et Délits (2012-2021)

Professeur : J. AGOUN

Sommaire

Phase 0 : Contexte, Problématique et Objectifs

Phase 1 : Analyse des Données Sources et Modélisation Relationnelle

Phase 2 : Analyse des limites du modèle relationnel & conception Graphe

Phase 3 : Migration des données vers le Modèle Graphe

Phase 4 : Validation et exploitation de la solution Graphe

Phase 5 : Et après ? Perspectives et Maintenance du Système

Conclusion générale

Annexes

Année universitaire : 2025-2026

Phase 0 : Contexte, Problématique et Objectifs

1.1. État des lieux et Contexte

En 2026, la réactivité des services de l'Etat dépend directement de leur capacité à exploiter la donnée massive. Actuellement, les informations relatives aux crimes et délits enregistrés par la Police Nationale (PN) et la Gendarmerie Nationale (GN) sur la période 2012-2021 sont sous-exploitées.

Ces données sont stockées dans un écosystème de fichiers Excel fragmentés en plusieurs feuilles (un fichier par année et par service), créant une défaillance technique majeure pour les analystes de la Direction Générale.

1.2. Analyse de la problématique

Le format actuel des données constitue un frein énorme à l'intelligence décisionnelle pour 3 raisons fondamentales :

1. La structure matricielle
2. La fragmentation (temporelle et de service)
3. Les erreurs humaines

D'abord, les fichiers sources sont conçus pour la lecture humaine. En effet, ce sont des tableaux croisés avec les infractions en lignes, et les départements en colonnes. Ce format est appelé "large". Son problème, c'est qu'il interdit toute requête SQL directe et rend les agrégations automatisées très complexes, voire impossibles.
Ensuite, l'absence de centralisation oblige les analystes à multiplier les manipulations manuelles pour obtenir une vision sur plusieurs années. Par exemple, le fait de vouloir comparer l'évolution d'un délit sur 10 ans dans un département nécessite l'ouverture et la fusion de plus de 20 sources distinctes.
Enfin, les BDD Excel ne permettent pas de modifier, ni d'ajouter des données sans risquer de faire des erreurs. Il est bien connu que les fichiers Excel sont pratiques, mais ils laissent place à beaucoup de manipulations manuelles, souvent par plusieurs personnes différentes et donc de potentielles erreurs.

La problématique centrale réside donc dans le fait de savoir comment est-ce que l'on peut passer d'un archivage Excel sous forme matricielle, siloté et absolument inexploitable ni requêtable, à une infrastructure de données unifiée et performante, capable de transformer des millions de données brutes en un réseau de données exploitables en temps réel, facilitant l'analyse des Crimes et Délits entre 2012 et 2021.

1.3. Objectifs Stratégiques

Pour répondre aux exigences du cahier des charges, nous allons passer par 4 phases.

D'abord, la phase 1, qui concerne l'analyse du fichier Excel source pour concevoir et implémenter une base de données relationnelle structurée (MCD/MLD) contenant les statistiques de criminalité.

Ensuite, la phase 2, qui vise à identifier les limites de performance du modèle relationnel et conception d'une structure en graphe (nœuds et relations).

Puis, la phase 3, avec la migration des données vers Neo4j à l'aide de scripts pour passer des tables aux entités connectées.

Enfin, la phase 4, avec l'Aide à la Décision et Performance. Cette dernière étape permettra de valider ou non l'intégrité des données, ainsi que l'exécution de requêtes métiers complexes pour comparer l'efficacité du graphe par rapport au relationnel.

Phase 1 : Analyse des données sources et modélisation relationnelle

Le succès d'un projet de migration vers une structure NoSQL ne se fait pas comme ça. Cela repose surtout sur une compréhension profonde des données ainsi que la sanctuarisation des données d'origine. Cette première phase détaille le passage de données brutes et hétérogènes à un entrepôt de données relationnel rigoureux, servant de "source de vérité" pour tout le projet.

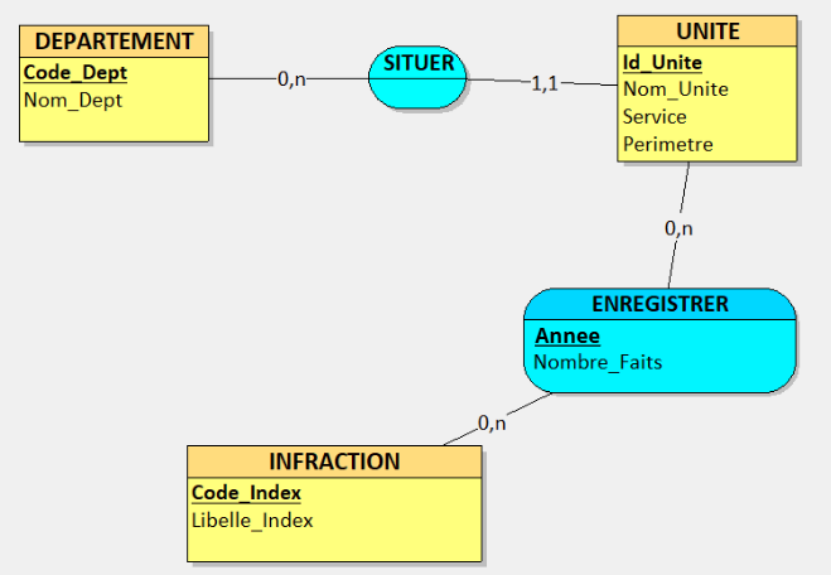
1.1. Diagnostic de l'existant : le chaos des tableurs

Les données initiales fournies par le Ministère de l'Intérieur ont révélé une fragmentation structurelle majeure. Bien que les statistiques soient exhaustives, leur stockage sous forme de 20 fichiers Excel distincts (répartis entre PN et GN par année) constitue une vraie "impasse analytique".
Le principal obstacle réside dans le format matriciel dit "Wide" (format large). Les départements sont en en-têtes de colonnes. Si ce format est simple pour la lecture humaine (qui prend du temps aussi…), il est inexploitable pour un système informatique (qui peut lui corréler des variables, effectuer des agrégations temporelles,...). De plus, l'absence de typage strict dans Excel favorise les erreurs d'intégrité (espaces insécables, doublons potentiels, formats numériques variables,...). L'objectif prioritaire a donc été de transformer ce chaos en une structure "Long Format", où chaque ligne représente un fait unique, daté et localisé.

1.2. La rigueur du Modèle 3NF

Pour structurer ces données, nous avons privilégié une architecture en 3ème Forme Normale (3NF) conçue via l'outil Looping. Looping est un outil pédagogique (qu'on a utilisé les années précédentes) qui permet de dessiner un MCD (entité, association, cardinalité,...), de générer automatiquement le MLD et de produire le script SQL correspondant. Il évite les erreurs de traduction entre modèle conceptuel et base réelle et est très pratique et simple à utiliser.

Modèle Conceptuel de Données (MCD) réalisé sous Looping



Le choix de la 3NF est crucial ici. Il permet d'éliminer la redondance (un libellé d'infraction comme "Vols simples" n'est stocké qu'une seule fois dans une table dédiée, évitant ainsi de répéter du texte lourd des millions de fois). Cela garantit également l'intégrité des données. Une correction sur un nom de service se répercute partout instantanément. Enfin, ce modèle assure l'évolutivité du système, permettant d'ajouter quand on le souhaite des données contextuelles, comme la population des départements, les régions,etc…, sans modifier les millions de lignes de statistiques existantes.

Choix des entités et sémantique

Le modèle s'articule autour de 4 piliers :

- **DEPARTEMENT** : Centralise l'info géographique
- **UNITE** : Représente le maillage territorial (Brigades/Commissariats).
- **INFRACTION** : Constitue le référentiel unique des types de crimes
- **ENREGISTRER** : Il s'agit de notre table de faits. C'est le coeur du système qui lie les unités aux infractions. Elle porte les mesures clés : l'année et le nombre de faits.

Concernant les cardinalités :
UNITE (1,1) → DEPARTEMENT : Une unité de PN/GN appartient physiquement à un seul et unique département.
DEPARTEMENT (0,n) → UNITE : Un département peut héberger plusieurs unités territoriales, ou aucune dans certains cas de zones spécifiques.
UNITE (0,n) → ENREGISTRER ← (0,n) INFRACTION : Il s'agit d'une relation "plusieurs-à-plusieurs". Une unité peut enregistrer plusieurs types d'infractions au fil des ans, et un même type de crime peut être constaté par de nombreuses unités différentes sur le territoire.

Ce modèle est le plus équilibré et sera notre point de départ pour une bonne migration vers le Graphe. Chaque entité deviendra un Noeud et l'association deviendra le Relation.

1.3. Mise en oeuvre technique : Le workflow ETL

Maintenant, afin de transformer les fichiers sources hétérogènes en une base exploitable, nous avons mis en place une chaîne de traitement automatisée (ETL).

L'usine de traitement : Jupyter Notebook

Nous avons choisi de développer un moteur en Python sous Jupyter Notebook. Plutôt que de manipuler manuellement 20 fichiers Excel au risque d'introduire des erreurs humaines, le script automatise l'extraction, la transformation et le chargement. Il parcourt l'ensemble des fichiers, identifie dynamiquement le service (PN/GN) et "dépivote" les colonnes pour passer au format tabulaire standard. Il permet principalement, en finalité, de migrer les données dans notre réceptacle DB Browser. Le lien GitHub vers le code utilisé est disponible en Annexe 1.

Notre réceptacle structuré : DB Browser (SQLite)

Si notre Notebook est l'intelligence du projet, DB Browser est notre mémoire. L'utilisation d'une étape intermédiaire en SQL n'est pas une perte de temps, mais une mesure de sécurité. Elle nous permet de définir un schéma strict (LDD) pour garantir l'intégrité. Une

statistique ne peut pas être insérée si elle n'est pas rattachée à une unité valide. Cette base est donc notre "Vérité" avant de passer à la migration vers Neo4J.
Pour construire ce réceptacle, nous avons simplement utilisé l'avantage de Looping, qui est la génération automatique du script SQL une fois notre MCD créé. Cependant, nous avons simplement modifié légèrement ce code manuellement pour répondre aux standards de DB Browser. Par exemple, le terme COUNTER (spécifique à Access et qui est donc généré sur Looping), a été remplacé par "AUTOINCREMENT". Ce choix de compteur garantit qu'il n'y aura pas de "conflit" d'ID lors de l'empilement des années successives. Une fois le script adapté et exécuté dans DB Browser, les tables étaient prêtes à recevoir les données "nettoyées" par notre moteur Python. Le script généré par Looping est disponible en Annexe 2.

1.4 Sanctuarisation SQL et gestion de l'unicité

La réussite de l'intégration repose sur certains choix. Initialement, nous avons envisagé une méthode simplifiée consistant à créer un fichier CSV par table (Département, Unités,..) pour les importer directement dans DB Browser, puis dans Neo4J. Cependant, cette méthode s'est avérée inefficace. Un import brut sans filtrage ni nettoyage aurait créé une base polluée avec beaucoup trop de nœuds et de relations, notamment en important les millions de lignes affichant "0" fait, polluant ainsi la lisibilité et l'efficacité du futur graphe.

C'est ici que l'automatisation via Python a démontré sa supériorité. L'automatisation via Python a permis de gérer finement les hétérogénéités (comme la ligne "Périmètre" spécifique à la Police) et de nettoyer les données numériques (suppression des espaces insécables d'Excel). Ce script a permis d'empiler des millions de lignes avec une précision que l'import manuel ne pouvait pas offrir, et surtout, en évitant les erreurs humaines.

Un point fondamental de cette étape a été la gestion de la redondance géographique. Dans les fichiers sources, une brigade de gendarmerie ou un commissariat apparaît autant de fois qu'il y a d'années étudiées. Pour éviter de dupliquer une unité dix fois en base, nous avons implémenté une contrainte d'intégrité directement dans le script de création de la table UNITE : la clause UNIQUE(Nom_Unite, Code_Dept).

Cette règle interdit à la base de créer deux fois la même unité pour un même département. Ce choix a deux conséquences majeures : d'abord, il sanctuarise le référentiel des services de l'État en SQL. Ensuite, il prépare une structure de graphe ultra-performant. En limitant le nombre de nœuds « Unité », nous transformons ce qui aurait été une pile de données isolées en une base relationnelle, où un nœud unique est relié à ses différentes statistiques annuelles. A noter que si nous avons conservé l'intégralité des données (y compris les zéros) dans SQLite, le filtrage des valeurs nulles sera réservé à l'étape de migration sur Neo4j pour optimiser les performances de parcours.

1.5 Qualité et Bilan de santé

A ce stade, nous avons donc créé les tables vides sur DB Browser, et le code a permis de traduire les 20 CSV pour importer les données dans les tables. Une fois cette importation terminée, nous avons également procédé à un enrichissement sémantique en injectant les noms réels des départements à partir de requêtes SQL (Update), permettant de passer d'une analyse technique (code "69") à une analyse métier ("Rhône") (voir fin de l'Annexe 2).

Par ailleurs, nous avons exécuté un script de diagnostic pour vérifier la "santé" de notre base de données. C'est une étape qui constitue la preuve finale de la réussite de la Phase 1. On ne pouvait évidemment pas se permettre de dire "le code a tout importé, c'est OK !", et avons donc eu la bonne conscience de ne pas poursuivre la migration tant que nous n'étions pas sûrs de la qualité de nos données à chaque étape.

```
=== BILAN DE SANTÉ DE LA BASE DE DONNÉES ===

Table DEPARTEMENT  : OK
Table UNITE        : OK
Table INFRACTION   : OK
Table ENREGISTRER  : OK

--------------------------------------------------
Table DEPARTEMENT  : 105 enregistrements
Table UNITE        : 1,482 enregistrements
Table INFRACTION   : 107 enregistrements
Table ENREGISTRER  : 1,139,015 enregistrements

--------------------------------------------------
✅ Pas de doublons dans UNITE (Nom_Unite + Code_Dept)

--------------------------------------------------
Unités sans données : 0 (doit être proche de 0)

--------------------------------------------------
✅ Volume total de faits enregistrés : 35,745,790

--------------------------------------------------
Années et services présents :
  2012 - GN
...
  2021 - GN
  2021 - PN
```

Les résultats de notre bilan sont très bons. En effet, nous avons centralisé 35 745 790 faits criminels, qui est effectivement la somme des faits des 20 feuilles du fichier Excel source. Grâce à la contrainte d'unicité, le nombre d'unités territoriales a été stabilisé à 1 482, garantissant un référentiel propre et sans doublons. Le bilan confirme également que 100% des unités sont rattachées à des données statistiques (0 unité orpheline), validant ainsi la cohérence totale de l'importation sur les 10 années étudiées.

1.6 Conclusion de la phase 1

A ce stade, la base de données Excel désordonnée et peu structurée est devenue un Data Warehouse relationnel propre et requêtable. Nous disposons désormais d'une source de vérité inattaquable. Cette rigueur préparatoire est indispensable, car elle nous permet désormais de filtrer stratégiquement les données (notamment en excluant les valeurs nulles) pour garantir une performance optimale lors de la migration vers le modèle graphe Neo4j.

Phase 2 : Analyse des limites du modèle relationnel & conception Graphe

Après avoir sanctuarisé nos 35 millions de faits dans un entrepôt SQLite, nous nous sommes confrontés à la réalité du terrain. Si le modèle relationnel est le “standard” pour le stockage de données, ça devient vraiment un frein dès lors qu’on cherche à explorer la donnée sous l’angle du réseau et de l’interconnexion.

2.1 Analyse des limites du modèle relationnel

Malgré la propreté de notre base 3NF, nous avons identifié trois obstacles majeurs qui limitent la capacité d’analyse des analystes.

1. Le coût des jointures

Dans une base SQL, les relations n’existent pas physiquement, elles sont calculées à la volée. Pour croiser un type d’infraction, un service (PN/GN) et une zone géographique sur 10 ans, le moteur doit comparer des index sur des millions de lignes via des opérations de JOIN.
Plus le volume de données augmente (avec l’intégration d’années futures par exemple), plus le temps de réponse s’allonge de manière non linéaire.

2. La “cécité réseau”

Le SQL est très bon dans les calculs verticaux (sommes, moyennes, …), mais il est mauvais horizontalement.
Par exemple, si le Ministère souhaite analyser si une hausse de la délinquance dans le Rhône influence les départements limitrophes (Isère, Ain,...), le SQL impose des auto-jointures (Self-Joins) qui sont assez compliqués. Le modèle relationnel ne possède pas de notion de proximité, la géographie est traitée comme une simple étiquette de texte, et non comme un territoire interconnecté.

3. Rigidité et maintenance du code

Toutes les évolutions de schéma (ici la structure tabulaire), comme l'ajout d'une nouvelle variable "Unité de nuit" par exemple, est coûteuse. En plus, les requêtes métier complexes se transforment très rapidement en une usine à gaz de 50 lignes beaucoup trop compliqué à gérer, difficile à maintenir et surtout source d'erreurs humaines pour les analystes.

2.2 Le changement de paradigme : l'approche NoSQL Graphe

Pour pallier ces limites, nous proposons un basculement vers Neo4j. Ici, l'information n'est plus stockée dans des cases (colonnes) mais sous forme d'objets vivants interconnectés.
Dans un graphe, les liens sont stockés physiquement sur le disque. Le fait de parcourir une relation ne coûte presque rien en calcul, car le système ne "cherche" pas la donnée, il suit juste le fil déjà existant.
Aussi, contrairement au SQL, le graphe est malléable. Nous pouvons enrichir le modèle avec de nouveaux types de liens (ex : [:EST_VOISIN_DE]) sans jamais avoir à reconstruire la structure des données existantes.

2.3 Proposition du modèle Graphe

Nous avons conçu un modèle sémantique où chaque entité de notre base SQL devient un Noeud (Node) et chaque lien logique devient une Relation (Relationship).

A - Définition des Noeuds (Labels)

- : **DEPARTEMENT** : Représente l'ancrage géographique
  → *Propriétés* : Code_Dept , Nom_Dept

- : **UNITE**: Représente le maillage territorial des forces de l'ordre
  → *Propriétés* : Nom_Unite , Service , Perimetre

- : **INFRACTION**: Représente la nomenclature officielle des crimes
  → *Propriétés* : Code_Index , Libelle_Index

B - Définition des Relations

- : **[:SITUE_DANS]** : Relie une Unité à son Département
  → *Structure* : (Unite)-[:SITUE_DANS]->(Departement)

- : **[:A_ENREGISTRE]** : C'est là que tout se joue. Contrairement au SQL, où l'info est perdue dans une table de pivot, ici, la relation devient intelligente.
  → *Structure* : (Unite)-[:A_ENREGISTRE]->(Infraction)
  → *Propriétés : Annee , Nombre_Faits*

2.4 Stratégie de migration

Notre processus de migration vers Neo4j ne sera pas une simple copie de la base SQL. Pour garantir une performance, nous appliquons à cette étape une règle de filtrage importante que nous avons déjà énoncé plus haut :

Toute relation [:A_ENREGISTRE] dont la propriété Nombre_Faits est égale à 0 ne sera pas créée.

On fait cela car dans tous les fichiers sources, les valeurs nulles représentent une part massive du volume global. En ne migrant que les faits réels, nous épurons le graphe pour ne visualiser que les connexions “actives”. Cela permet de gagner en clarté visuelle lors des analyses, et surtout d’optimiser la mémoire vive du serveur Neo4j en évitant de stocker de l’information vide.

Phase 3 : Migration des données vers le Modèle Graphe

Cette phase est donc le coeur du projet, où nous devons marquer la transition concrète du monde relationnel (tableaux) vers le monde des graphes (réseaux dynamiques). L’objectif est donc de transformer nos 35 millions de lignes de statistiques en un écosystème où la relation entre les entités prime sur le stockage individuel.

3.1 Une migration “intelligente” et sélective

Plutôt que de réaliser un import brut qui aurait recréé l'intégralité des tables SQL sous forme de noeuds, incluant des millions de données vides, nous avons opté pour une migration sémantique pilotée par un script Python.

Cette approche repose sur 3 principes :

1. **L’extraction sélective :**

Seules les dimensions essentielles ont été extraites de SQLite (nom d’unité, type d’infraction,...) pour éviter de surcharger le graphe avec des clés techniques (ID_SQL).

2. **Le filtrage à la source :**

Pour optimiser les performances de Neo4j et réduire le “bruit”, nous avons appliqué comme dit plus haut, un filtre strict, où les lignes affichant 0 fait ne sont pas migrées. Cela évite la création de “liens fantômes” et garantit que chaque relation du graphe représente une activité réelle, et pas juste des 0.

3. **La transformation en temps réel :**

Grâce au driver Python, les entités statiques sont devenues des Noeuds, tandis que les données d'activité (statistiques annuelles) ont été transformées en Relations enrichies de propriétés.

3.2 Implémentation technique

Le script développé utilise le protocole Bolt pour communiquer directement avec l'instance Neo4j. Bien sûr, l'IA Gemini de Google nous a permis de le rendre fonctionnel plus rapidement et surtout d'être optimisée.
Cela aide donc à assurer la rapidité du traitement sur presque 600 000 relations, en utilisant la stratégie de Batching avec la clause "UNWIND".

Points clés du script :

- **Contrainte d'unicité :**

Avant l'import, le script sécurise la base en créant des index uniques sur les codes départements et infractions.

- **Mécanisme MERGE :**

Le script utilise systématiquement le MERGE pour éviter les doublons si le script est relancé.

- **Optimisation par lots (Batching) :**

Les faits sont envoyés par paquets de 1000 via une liste d'objets Python, réduisant drastiquement les aller retours réseau et la consommation CPU du serveur Neo4J.

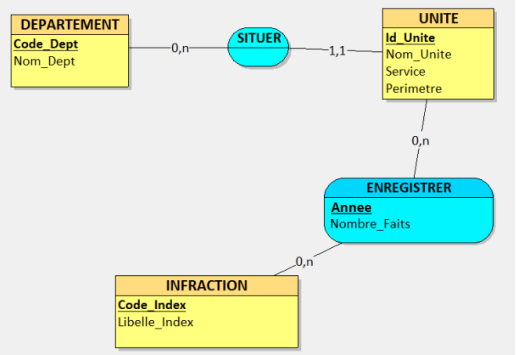
3.3 Vérification de la conformité du modèle (Validation)

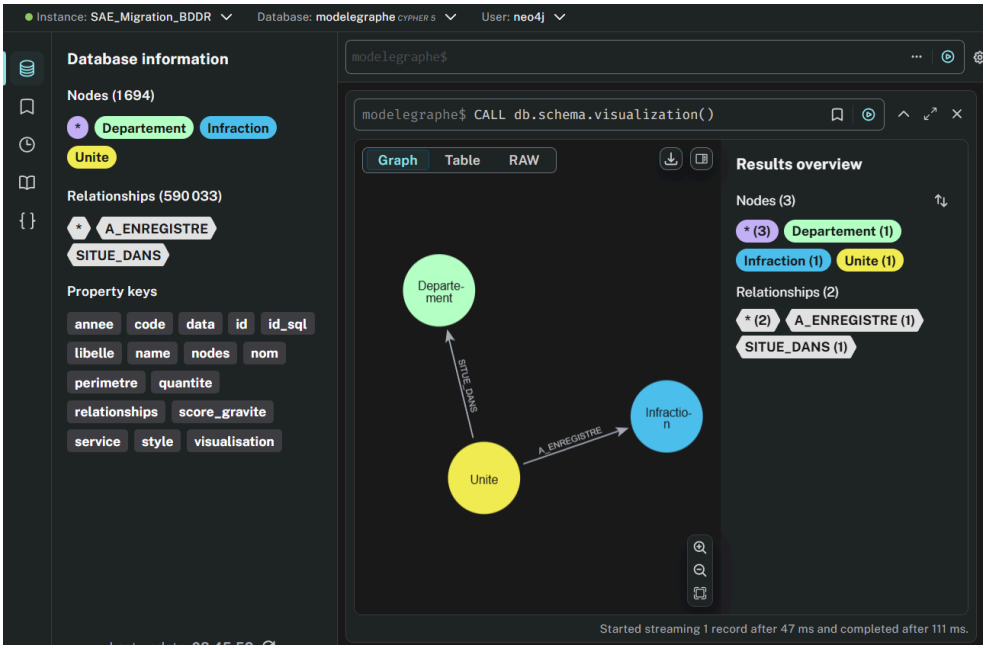
Pour s'assurer que notre architecture théorique est bien la même en pratique, nous avons soumis la base à une double validation.

A. Validation visuelle du schéma

Grâce à la commande "CALL db.schema.visualization()" sur Neo4j, nous avons généré la structure réelle de la base.

On voit bien que le graphe ci-après respecte notre MCD, dont le rappel est ci-dessous.

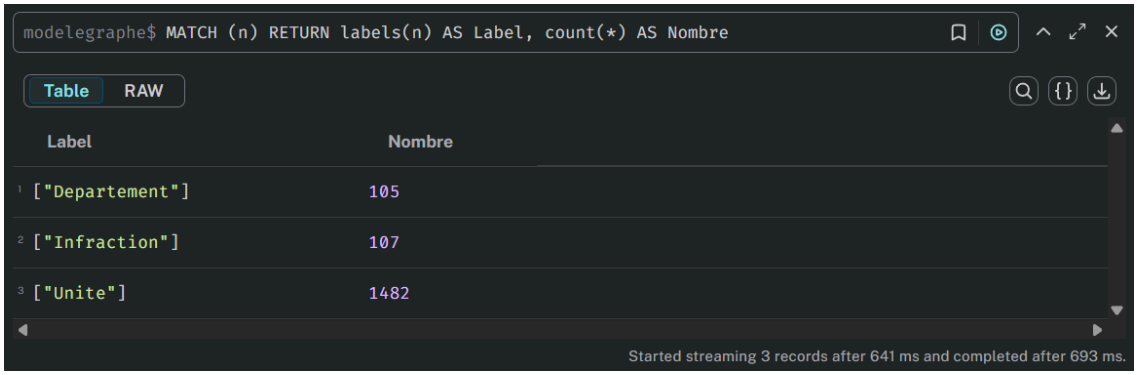




On retrouve exactement nos 3 labels (UNITE, DEPARTEMENT et INFRACTION), et les 2 types de relations attendus (SITUE_DANS, A_ENREGISTRE). Cette visualisation confirme l'absence de "noeuds parasites", donc chaque flèche respecte la logique métier.
On voit également que les 1694 noeuds sont bien la somme des noeuds que l'on détaille juste après.

B. Validation visuelle du schéma

Le bilan automatique généré par la commande "`MATCH (n) RETURN labels(n) AS Label, count(*) AS Nombre`" atteste de l'intégrité des données.



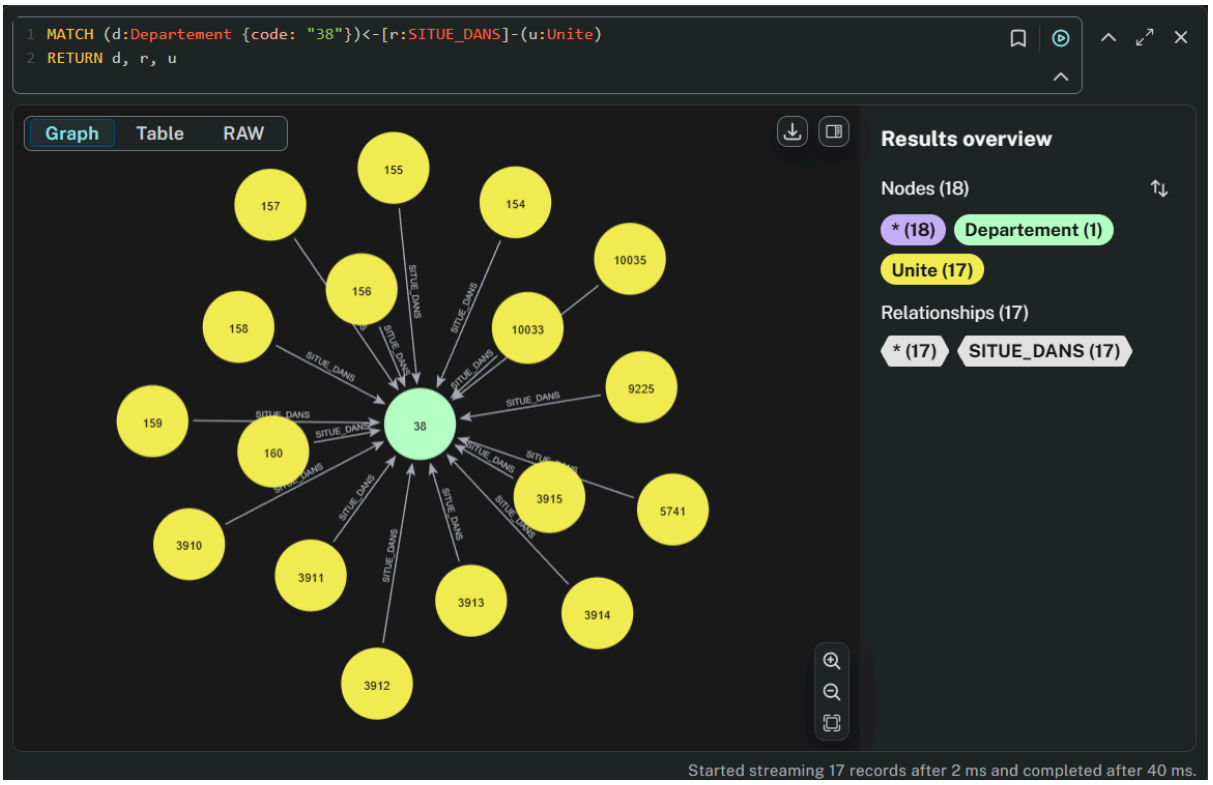
Nous retrouvons bien nos :

- 107 noeuds Infraction, qui est le nombre que l'on retrouvait dans l'excel concernant la nomenclature des types d'infractions.
- 105 noeuds Departement, qui représente la couverture totale du territoire (+ DOMTOM)
- 1482 noeuds Unite, qui représente le référentiel du nombre d'unités dédoublonné ! 1 force de l'ordre peu importe l'année.
- environ 600k relation A_ENREGISTRE, qui représente l'historique complet et épuré (pas les 0) des faits constatés sur 10 ans.

3.4 Preuves visuelles de l’intégrité structurelle

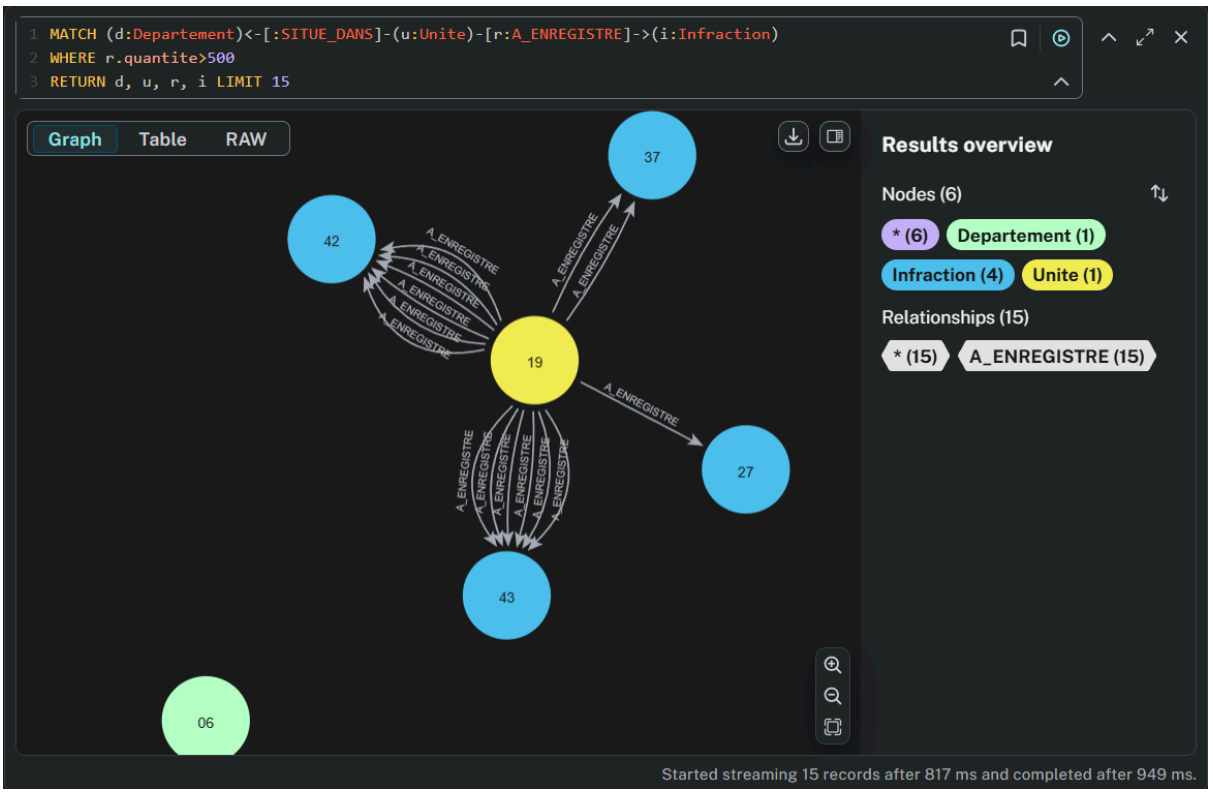
Pour valider visuellement la cohérence de notre architecture finale, nous avons réalisé des captures en conditions réelles. Ces visuels confirment que le modèle théorique a été fidèlement traduit.

1. Le test de l’Ancrage Territorial



Ce test vérifie que le maillage géographique est respecté. En isolant le département de l’Isère (38), la visualisation révèle une topologie en étoile parfaite.
On observe un noeud central vert (le département) vers lequel convergent toutes les unités rattachées (noeuds jaunes).
Cela nous confirme que les 17 unités filtrées pour cette vue sont correctement ancrées à leur parent géographique via la relation “SITUE_DANS”. Il n’existe aucun orphelin flottant sans parent géographique. On l’a d’ailleurs vu dans notre bilan de santé tout à l’heure.

2. Le test du Triplet Sémantique (Trinité de l'information)



Ce second test valide quant à lui la trinité de notre modèle, à savoir l'acteur, le lieu et l'acte. Au centre, une unité (noeud jaune) est simultanément liée à son département d'origine (noeud vert) et à ses différentes catégories d'infractions (noeuds bleus). La flèche du lien entre l'unité et le département ne s'affiche pas mais c'est simplement une erreur de visualisation due à la limite de 15.
On observe donc que chaque flèche porte physiquement les propriétés de l'année et de la quantité. Ce test prouve que le graphe n'est pas juste une carte, mais une base multidimensionnelle capable de restituer l'activité criminelle précise d'un point géographique donné.

3.5 Conclusion de la migration

La migration vers Neo4j est un succès validé par les chiffres et les images. Nous disposons maintenant d'un environnement épuré de 590 033 relations actives, débarrassé du "bruit" des valeurs nulles qui encombraient la base SQLite. Cette structure optimisée, visible dans la console d'information de la base, est désormais prête pour la phase finale, qui est la validation et l'exploitation de notre solution graphe.

Phase 4 : Validation et exploitation de la solution Graphe

Cette phase constitue la phase “juge” de notre SAE. Son objectif est d’une part de certifier que la migration n’a altéré aucune de nos données (Intégrité), mais aussi de démontrer que le modèle Graphe, ainsi que la migration que l’on a faite, surpasse le modèle relationnel dans l’exploitation quotidienne des services des forces de l’ordre (Performance).

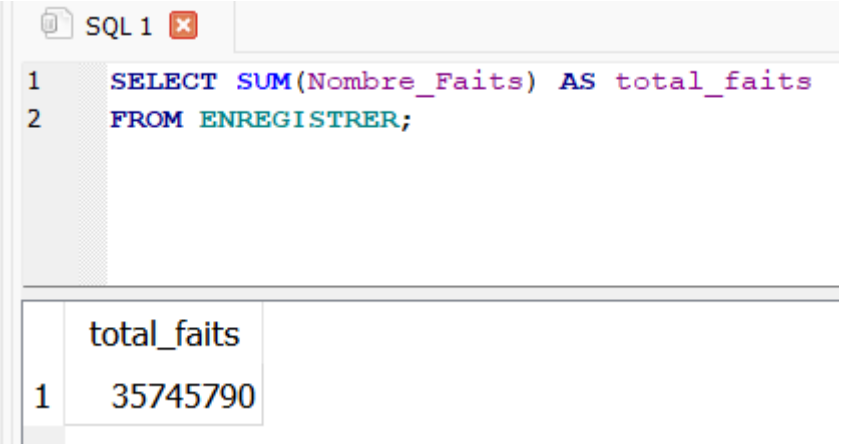
4.1 Vérification de la cohérence des données massives

Avant toute analyse métier, nous avons fait quelques “examens” pour garantir que le graphe est le miroir exact de notre source SQLite.

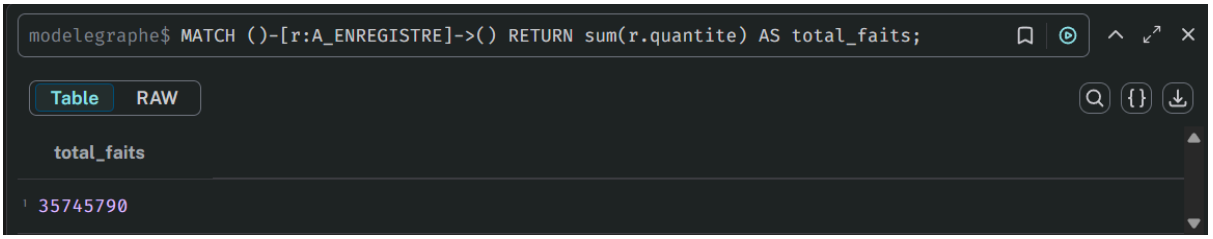
A. L’audit de masse

Nous avons comparé la somme totale des faits enregistrés entre les deux environnements.

En SQL (DB Browser) :



En Cypher (Neo4j) :



Les deux bases affichent strictement 35 745 790 faits. Cette égalité parfaite valide que notre moteur de migration Python a transféré l’intégralité de la matière première sans perte ni doublon. De plus, c’est exactement le nombre qu’on souhaitait voir étant donné que c’est la somme des 20 CSV.

B. L'audit structurel

Nous avons vérifié aussi la solidité du maillage territorial. En graphe, une unité sans département est une anomalie structurelle.
Pour contrôler cela :

En Cypher (Neo4j) :

```
modelegraphe$ MATCH (u:Unite) WHERE NOT (u)-[:SITUE_DANS]->(:Departement) RETURN count(u)
```

Table RAW

count(u)
0

Le 0 confirme donc que 100% de notre maillage (les 1482 unités) est correctement ancré géographiquement et qu'il n'y a pas d'unité "non liée".

4.2 Justesse et exactitude des requêtes métiers

Nous avons aussi testé la capacité du Graphe à répondre aux questions que les analystes du Ministère auraient à poser.

A. Identification des types de crimes par département

D'abord, le défi est qu'il faut traverser 3 niveaux de données : **Département** → **Unité** → **Infraction**.
Par exemple pour le Rhône (et une limite à 5) :

En Cypher (Neo4j) :

```
MATCH (d:Departement {code: "69"})<-[:SITUE_DANS]-(u:Unite)-[r:A_ENREGISTRE]->(i:Infraction)
RETURN i.libelle AS Crime, sum(r.quantite) AS Total
ORDER BY Total DESC LIMIT 5
```

Table RAW

Crime	Total
"Autres vols simples contre des particuliers dans des locaux ou lieux publics"	156475
"Vols à la roulotte"	116878
"Cambriolages de locaux d'habitations principales"	87516
"Autres coups et blessures volontaires criminels ou correctionnels"	76779
"Escroqueries et abus de confiance"	69666

Started streaming 5 records after 502 ms and completed after 789 ms.

En SQLite (DB Browser) :

SQL 1

```sql
SELECT
    i.Libelle_Index AS Crime,
    SUM(e.Nombre_Faits) AS Total
FROM ENREGISTRER e
JOIN UNITE u
    ON e.Id_Unite = u.Id_Unite
JOIN INFRACTION i
    ON e.Code_Index = i.Code_Index
JOIN DEPARTEMENT d
    ON u.Code_Dept = d.Code_Dept
WHERE d.Code_Dept = '69'
GROUP BY i.Libelle_Index
ORDER BY Total DESC
LIMIT 5;
```

	Crime	Total
1	Autres vols simples contre des particuliers dans des locaux ou lieux publics	156475
2	Vols à la roulotte	116878
3	Cambriolages de locaux d'habitations principales	87516
4	Autres coups et blessures volontaires criminels ou correctionnels	76779
5	Escroqueries et abus de confiance	69666

```
L'exécution s'est terminée sans erreur.
Résultat : 5 enregistrements ramenés en 3683ms
```

On voit bien que là où le SQL oblige à penser en colonnes, en clés, et grosses jointures, Cypher permet de dessiner simplement le cheminement. On voit également que le temps de “requêtage” est de moins d’une seconde avec Neo4j et son modèle Graphe contre plus de 3,5 secondes pour le modèle relationnel sur DB Browser.
La justesse des résultats (qui sont identiques) valide la fiabilité sémantique de nos relations.

B. Analyse de la densité du maillage

Nous avons vérifié si la répartition des unités par département correspond à la réalité du terrain.

En Cypher (Neo4j) :

```cypher
MATCH (d:Departement)<-[:SITUE_DANS]-(u:Unite)
RETURN d.nom, count(u) AS Nb_Unites
ORDER BY Nb_Unites DESC LIMIT 5
```

Table RAW

	d.nom	Nb_Unites
1	"Paris"	87
2	"Hauts-de-Seine"	55
3	"Rhône"	49
4	"Nord"	47
5	"Bouches-du-Rhône"	43

En SQLite (DB Browser) :

SQL 1

```sql
SELECT
    d.Nom_Dept,
    COUNT(u.Id_Unite) AS Nb_Unites
FROM DEPARTEMENT d
JOIN UNITE u
    ON u.Code_Dept = d.Code_Dept
GROUP BY d.Nom_Dept
ORDER BY Nb_Unites DESC
LIMIT 5;
```

	Nom_Dept	Nb_Unites
1	Paris	87
2	Hauts-de-Seine	55
3	Rhône	49
4	Nord	47
5	Bouches-du-Rhône	43

Cette requête permet de valider instantanément la cohérence démographique de notre base Graphe, car le TOP 5 des départements les plus denses ressortent avec le même nombre d'unités.

4.3 Bonus : Le duel de performances (SQL vs Graphe)

C'est dans cette partie que la supériorité de Neo4j devient flagrante. Nous avons ici comparé la "charge mentale", la performance et la complexité technique pour une requête identique.

La problématique ici est d'extraire l'activité précise du CGD LA MURE pour l'année 2021.

Code Cypher (Neo4j) :

```cypher
MATCH (u:Unite)-[r:A_ENREGISTRE]->(i:Infraction)
WHERE u.nom CONTAINS "CGD LA MURE" AND r.annee = 2021
RETURN i.libelle, r.quantite
ORDER BY r.quantite DESC
```

Code SQLite (DB Browser) :

```sql
SELECT i.Libelle_Index, e.Nombre_Faits
FROM ENREGISTRER e
JOIN INFRACTION i ON e.Code_Index = i.Code_Index
JOIN UNITE u ON e.Id_Unite = u.Id_Unite
WHERE u.Nom_Unite LIKE '%CGD LA MURE%' AND e.Annee = 2021
ORDER BY Nombre_Faits DESC;
```

En Cypher (Neo4j) :

```
MATCH (u:Unite)-[r:A_ENREGISTRE]->(i:Infraction)
WHERE u.nom CONTAINS "CGD LA MURE" AND r.annee = 2021
RETURN i.libelle, r.quantite
ORDER BY r.quantite DESC
```

Table RAW

	i.libelle	r.quantite
1	"Escroqueries et abus de confiance"	302
2	"Vols à la roulotte"	276
3	"Autres vols simples contre des particuliers dans des locaux ou lieux publics"	233
4	"Autres coups et blessures volontaires criminels ou correctionnels"	190
5	"Autres vols simples contre des particuliers dans deslocaux privés"	164
6	"Cambriolages de locaux d'habitations principales"	134
7	"Camb.de locaux industriels, commerciaux ou financiers"	122
8	"Autres délits"	114
9	"Menaces ou chantages dans un autre but"	109
10	"Atteintes à la dignité et à la personnalité"	91

Started streaming 67 records after 814 ms and completed after 956 ms.

En SQLite (DB Browser) :

SQL 1

```
SELECT i.Libelle_Index, e.Nombre_Faits
FROM ENREGISTRER e
JOIN INFRACTION i ON e.Code_Index = i.Code_Index
JOIN UNITE u ON e.Id_Unite = u.Id_Unite
WHERE u.Nom_Unite LIKE '%CGD LA MURE%' AND e.Annee = 2021
ORDER BY Nombre_Faits DESC;
```

	Libelle_Index	Nombre_Faits
1	Escroqueries et abus de confiance	302
2	Vols à la roulotte	276
3	Autres vols simples contre des ...	233
4	Autres coups et blessures volontaires...	190
5	Autres vols simples contre des ...	164
6	Cambriolages de locaux d'habitations...	134
7	Camb.de locaux industriels, ...	122
8	Autres délits	114
9	Menaces ou chantages dans un autre...	109
10	Atteintes à la dignité et à la ...	91
11	Délits au sujet de la garde des mineurs	83
12	Vols d'automobiles	81
13	Violences, mauvais traitements et ...	80
14	Usage de stupéfiants	69
15	Autres vols simples contre des ...	60

L'exécution s'est terminée sans erreur.
Résultat : 107 enregistrements ramenés en 465ms

Ici, ce n’est pas forcément en temps que le modèle graphe est meilleur, car SQLite n’est pas récent et est donc très bien optimisé, mais plutôt en maniabilité et compréhension pour l’analyste. On voit que le code pour parvenir à la même finalité n’est pas de la même difficulté. En SQLite, il faut gérer des clés primaires/étrangères (Code_Index, Id_Unite) et faire plusieurs JOIN. Si on oublie une jointure, le résultat est faux ou la base plante.
En Neo4j, il n’y a aucune difficulté. On dessine littéralement la relation avec les parenthèses et les flèches (u)-[r]->(i). C'est du "langage parlé".

Pour donner un autre exemple plus complexe, on peut essayer d’identifier toutes les unités du département du Rhône dont le volume de “Cambriolages” en 2021 est strictement supérieur à la moyenne des unités de ce même département pour ce même crime. Cette requête nécessite beaucoup de calculs (moyenne, filtrage, comparaison à la moyenne).

Code Cypher (Neo4j) :

```
MATCH (d:Departement {code:"69"})<-[:SITUE_DANS]-(u:Unite)-[r:A_ENREGISTRE {annee:2021}]->(i:Infraction)
WHERE i.libelle CONTAINS "Cambriolage"
WITH avg(r.quantite) AS moyenne

MATCH (d:Departement {code:"69"})<-[:SITUE_DANS]-(u:Unite)-[r:A_ENREGISTRE {annee:2021}]->(i:Infraction)

WHERE i.libelle CONTAINS "Cambriolage" AND r.quantite > moyenne

RETURN u.nom AS Unite, r.quantite AS Cambriolages
ORDER BY Cambriolages DESC
```

Code SQLite (DB Browser) :

```
SELECT u.Nom_Unite, e.Nombre_Faits
FROM ENREGISTRER e
JOIN UNITE u ON e.Id_Unite = u.Id_Unite
JOIN INFRACTION i ON e.Code_Index = i.Code_Index
WHERE u.Code_Dept = '69'
AND e.Annee = 2021
AND i.Libelle_Index LIKE '%Cambriolage%'
AND e.Nombre_Faits > (

   SELECT AVG(e2.Nombre_Faits)
   FROM ENREGISTRER e2
   JOIN UNITE u2 ON e2.Id_Unite = u2.Id_Unite
   JOIN INFRACTION i2 ON e2.Code_Index = i2.Code_Index
   WHERE u2.Code_Dept = '69'
   AND e2.Annee = 2021
   AND i2.Libelle_Index LIKE '%Cambriolage%'
)
ORDER BY e.Nombre_Faits DESC;
```

En Cypher (Neo4j) :

```
MATCH (d:Departement {code:"69"})<-[:SITUE_DANS]-(u:Unite)-[r:A_ENREGISTRE {annee:2021}]->
(i:Infraction)
WHERE i.libelle CONTAINS "Cambriolage"

WITH avg(r.quantite) AS moyenne

MATCH (d:Departement {code:"69"})<-[:SITUE_DANS]-(u:Unite)-[r:A_ENREGISTRE {annee:2021}]->
(i:Infraction)
WHERE i.libelle CONTAINS "Cambriolage" AND r.quantite > moyenne

RETURN u.nom AS Unite, r.quantite AS Cambriolages
ORDER BY Cambriolages DESC
```

Table RAW

	Unite	Cambriolages
1	"LYON AGGLOMERATION"	5054
2	"LYON AGGLOMERATION"	2953
3	"CGD L ARBRESLE"	660
4	"CGD GIVORS"	597
5	"CGD BRON"	499
6	"CGD VILLEFRANCHE SUR SAONE"	403

En SQLite (DB Browser) :

```
SELECT u.Nom_Unite, e.Nombre_Faits
FROM ENREGISTRER e
JOIN UNITE u ON e.Id_Unite = u.Id_Unite
JOIN INFRACTION i ON e.Code_Index = i.Code_Index
WHERE u.Code_Dept = '69'
AND e.Annee = 2021
AND i.Libelle_Index LIKE '%Cambriolage%'
AND e.Nombre_Faits > (

    SELECT AVG(e2.Nombre_Faits)
    FROM ENREGISTRER e2
    JOIN UNITE u2 ON e2.Id_Unite = u2.Id_Unite
    JOIN INFRACTION i2 ON e2.Code_Index = i2.Code_Index
    WHERE u2.Code_Dept = '69'
    AND e2.Annee = 2021
    AND i2.Libelle_Index LIKE '%Cambriolage%'
)

ORDER BY e.Nombre_Faits DESC;
```

	Nom_Unite	Nombre_Faits
1	LYON AGGLOMERATION	5054
2	LYON AGGLOMERATION	2953
3	CGD L ARBRESLE	660
4	CGD GIVORS	597
5	CGD BRON	499
6	CGD VILLEFRANCHE SUR SAONE	403
7	CGD LYON	354
8	CIAT DE VILLEFRANCHE SUR SAONE	216

En SQL, cette opération impose des sous-requêtes complexes. Le moteur doit passer sur la table de faits 2 fois, une fois pour la moyenne, une seconde fois pour filtrer les unités.
Il faut donc gérer deux blocs de lecture et s'assurer que les jointures sont strictement identiques pour ne pas fausser les résultats. Il y aussi une boucle imbriquée, donc plus le nombre d'unités augmente, plus le coût de comparaison des index croît de façon exponentielle.

Côté Neo4j, la requête Cypher est linéaire. On récupère les données, on calcule la moyenne "à la volée", et on continue le cheminement sans jamais ré-explorer la base depuis le début. On suit donc un flux logique de données, facile à écrire, et l'avantage est qu'une fois le noeud "Rhône" identifié, il se contente de suivre les pointeurs physiques vers les unités. Il n'y a aucun calcul de correspondance de clés (JOIN) durant la traversée.

Ce duel démontre que le modèle Graphe n'est pas forcément, ni seulement plus rapide, il est plus intelligent. Il permet de réaliser des analyses contextuelles et comparatives qui seraient soit trop lentes, soit trop complexes à maintenir dans une architecture relationnelle classique. Nous passons d'une base de données de stockage à une base de données de compréhension.

4.4 Synthèse : Pourquoi le graphe l'emporte ?

A la fin de nos tests, 3 choses ressortent et justifient le passage du Graphe pour le Ministère de l'Intérieur. D'abord, la performance brute, l'agilité de développement, ainsi que la flexibilité "Réseau".

La performance brute, car en SQL, plus la base grandit, plus les jointures ralentissent le système. Dans Neo4j, passer d'une unité à un crime est quasi instantané, que la base contienne 1 million ou 1 milliard de faits.

L'agilité de développement, car un analyste peut rédiger une requête complexe en 2 minutes sur Neo4j, là où un expert SQL passerait 20 minutes à débugger des jointures.

La flexibilité "Réseau", car si demain le Ministère souhaite analyser la propagation de la délinquance entre départements voisins, il suffit de créer une relation [:EST_VOISIN_DE]. En SQL, cela exigerait une restructuration lourde et coûteuse des tables et requêtes existantes.

4.5 Conclusion de la Phase 4

En résumé, cette phase de migration et de tests ne s'est pas contentée de valider des chiffres. Elle a marqué un vrai changement d'environnement pour l'exploitation des données du Ministère.

Au-delà de la technique, c'est l'expérience de l'analyste qui est totalement transformée. Là où le modèle relationnel (SQLite) nous obligeait à "penser en tableaux" et à construire des jointures de plus en plus lourdes, Neo4j nous permet de suivre l'instinct de l'analyste.

Concrètement, on ne calcule plus des liens, on parcourt des connexions physiques déjà en place. En passant de 15 lignes de SQL complexe à 7 lignes de Cypher limpide, on réduit aussi le risque d'erreurs humaines. Il y a donc moins ce côté "je dois être un expert de la BDD pour poser des questions complexes".

Phase 5 : Et après ? Perspectives et Maintenance du Système

Cette dernière phase permet juste de compléter la documentation technique en ouvrant sur l'évolutivité de la solution.

5.1. Stratégies de mise à jour : L'ajout de nouvelles données

Un système d'information est utile seulement s'il est vivant. Voici comment intégrer les futures statistiques (années futures):

Sur la base relationnelle SQLite, il suffit de relancer le script Python "migration_sqlite.py" disponible sur mon GitHub avec les nouveaux fichiers CSV dans le dossier source. Grâce à la contrainte UNIQUE(Nom_Unite, Code_Dept), le script ne créera pas de doublons d'unités mais empilera simplement les nouvelles lignes dans la table ENREGISTRER.

Sur la base graphe Neo4j, c'est la même procédure, mais avec le script "Migration_SQLite_vers_Neo4j". Le script utilise la clause MERGE. Si une unité ou un département existe déjà, Neo4j ne fera que créer la nouvelle relation [:A_ENREGISTRE] pour l'année concernée. On peut rejouer les scripts sans risque de polluer la base.

5.2. Les autres méthodes de migration

Dans ce projet, nous avons privilégié une migration pilotée par Python pour sa précision sur le filtrage des zéros. Il existe toutefois d'autres méthodes que le Ministère pourrait envisager selon le contexte :

1. **L'import natif (LOAD CSV) :** C'est la méthode la plus simple de Neo4j. Elle est rapide pour des fichiers parfaits, mais elle manque de souplesse pour le nettoyage complexe (comme nos espaces insécables Excel) et peut créer une explosion de nœuds (c'est d'ailleurs ce qui m'est arrivé la première fois).
2. **L'outil Neo4j ETL Tool :** Une interface graphique qui se connecte directement à SQLite pour transformer les tables en nœuds. C'est très efficace pour une migration "miroir", mais cela ne permet pas le filtrage sémantique fin que nous avons appliqué (exclusion des faits nuls).
3. **L'utilisation d'APOC (Procédures stockées) :** Pour des flux de données en temps réel, on utiliserait les bibliothèques APOC pour charger des données JSON ou JDBC directement depuis des serveurs de production.

5.3 Exemple d'ajout d'un attribut (Score de Gravité)

On peut d'ailleurs faire le test en ajoutant par exemple un attribut dans les 2 environnements.

Scénario : Le Ministère souhaite attribuer un "Score de Gravité" (de 1 à 10) aux infractions pour prioriser les interventions.

- **En SQL (DB Browser) : Une modification structurelle lourde**

Action : Il faut modifier la structure physique de la table.

ALTER TABLE INFRACTION ADD COLUMN Score_Gravite INTEGER DEFAULT 1;

Mise à jour : `UPDATE INFRACTION SET Score_Gravite = 10 WHERE Libelle_Index LIKE '%omicides%';`

Table : INFRACTION — Filtrer dans n'importe quelle colonne

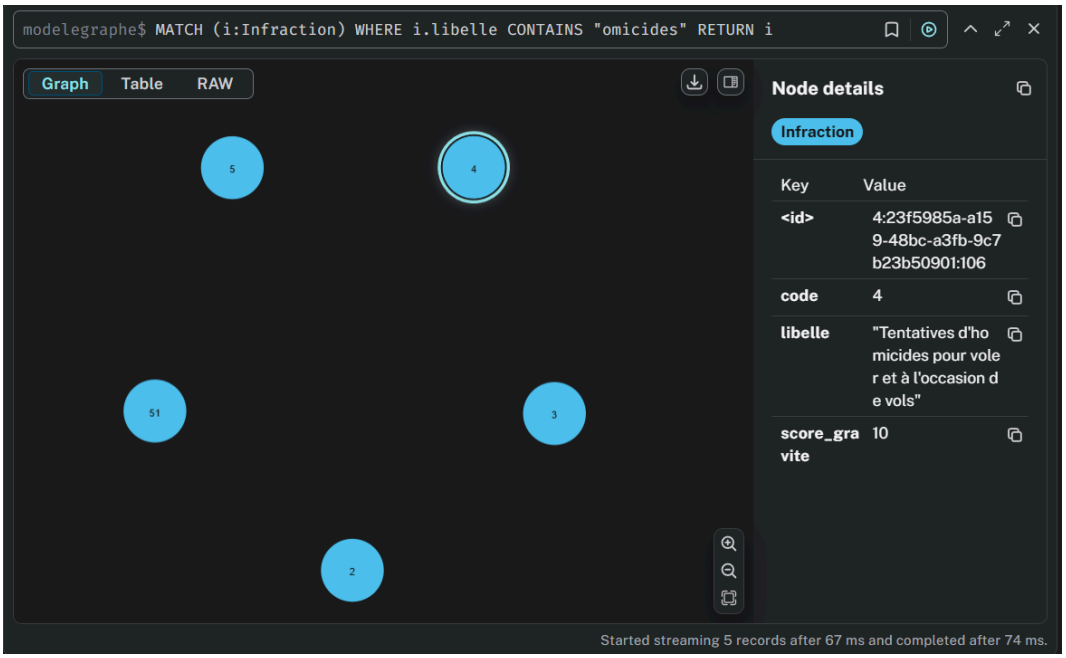
	Code_Index	Libelle_Index	Score_Gravite
	Filtre	Filtre	Filtre
1	2	Homicides pour voler et à l'occasion de vols	10
2	3	Homicides pour d'autres motifs	10
3	4	Tentatives d'homicides pour voler et à l'occasion de vols	10
4	5	Tentatives homicides pour d'autres motifs	10
5	51	Homicides commis contre enfants de moins de 15 ans	10
6	1	Règlements de compte entre malfaiteurs	1
7	6	Coups et blessures volontaires suivis de mort	1

Le problème : L'opération ALTER TABLE sur une base de millions de lignes peut verrouiller la table ("Lock"), rendant la base indisponible pendant le traitement. De plus, cela crée une colonne pour toutes les lignes, même si l'information n'est pertinente que pour quelques-unes, ici les homicides (consommation inutile d'espace).

En Graphe (Neo4j) : Une évolution "organique"

Action : On injecte la donnée directement là où elle est utile.

MATCH (i:Infraction) WHERE i.libelle CONTAINS "omicides" SET i.score_gravite = 10



On peut voir que le score de gravité est bien de 10 quand le libellé contient le mot “Homicides”.

L'avantage, c'est le principe du Schema-less. Neo4j n'a pas besoin de "prévenir" la base de données. Seuls les nœuds concernés reçoivent la propriété. Il n'y a aucun verrouillage de la base et aucune perte d'espace pour les autres nœuds.

Conclusion générale

Ce projet marque une vraie étape dans la modernisation de l'analyse des données de sécurité nationale. En passant d'un archivage Excel fragmenté à un écosystème de données interconnecté, nous avons transformé une simple pile de chiffres en un véritable outil d'intelligence stratégique. L'enseignement majeur de cette mission est que la performance technique ne suffit pas, mais que c'est l'agilité qui prime. Si le modèle relationnel a été indispensable pour assainir et sanctuariser" nos 35,7 millions de faits, il agissait comme un plafond pour l'analyse complexe. En migrant vers Neo4j, nous n'avons pas seulement gagné en rapidité d'exécution, nous avons libéré l'analyste des contraintes du "penser en tableaux".

Désormais, interroger la base de données revient à suivre l'instinct de l'analyste. On ne calcule plus des jointures coûteuses, on parcourt des relations physiques déjà tracées. Cette approche "orientée graphe" permet de passer de 15 lignes de code SQL indigeste à une simple flèche en Cypher, réduisant ainsi drastiquement le risque d'erreur humaine tout en multipliant les capacités de découverte de patterns criminels.

Au-delà de la réussite technique, cette architecture est une rampe de lancement vers l'avenir. Le Ministère dispose maintenant d'une base saine, épurée et prête pour la “Graph Data Science”. Qu'il s'agisse de détecter des réseaux de délinquance itinérante ou d'anticiper la propagation de certains délits entre départements voisins, cet outil offre une vision "réseau" que le format tabulaire ne pourra jamais égaler. Nous avons fait le choix de la clarté et de l'évolutivité pour répondre aux défis de sécurité de demain.

Annexe 1 : Lien vers GitHub

https://github.com/MathiasJnn/BUT-SD-3-migration-crimes-neo4j

Annexe 2 : Script SQL généré par Looping pour la création des tables sur DB Browser :

```
-- 1. Table DEPARTEMENT
CREATE TABLE DEPARTEMENT(
  Code_Dept VARCHAR(3),
  Nom_Dept VARCHAR(50),
  PRIMARY KEY(Code_Dept)
);

-- 2. Table UNITE
CREATE TABLE UNITE(
  Id_Unite INTEGER PRIMARY KEY AUTOINCREMENT, -- PK déclarée ici
  Nom_Unite VARCHAR(150),
  Service VARCHAR(2) NOT NULL,
  Perimetre VARCHAR(50),
  Code_Dept VARCHAR(3) NOT NULL,
  UNIQUE(Nom_Unite, Code_Dept),
  FOREIGN KEY(Code_Dept) REFERENCES DEPARTEMENT(Code_Dept)
);

-- 3. Table INFRACTION
CREATE TABLE INFRACTION(
  Code_Index INTEGER PRIMARY KEY,
  Libelle_Index VARCHAR(255) NOT NULL
);

-- 4. Table de faits ENREGISTRER
CREATE TABLE ENREGISTRER(
  Id_Unite INT,
  Code_Index INT,
  Annee INT,
  Nombre_Faits INT,
  PRIMARY KEY(Id_Unite, Code_Index, Annee),
  FOREIGN KEY(Id_Unite) REFERENCES UNITE(Id_Unite),
  FOREIGN KEY(Code_Index) REFERENCES INFRACTION(Code_Index)
);
```

+ *exemple de la ligne de code SQL ajouté pour imputer le nom de chaque département (ici l'Ain) :* ***UPDATE DEPARTEMENT SET Nom_Dept = 'Ain' WHERE Code_Dept = '01';***